软件安装说明书

一、安装环境

由于本人资源有限，因此以下均为都是测试环境虚拟机单机部署配置。

硬件环境：

中央处理器：AMD R9 7940hx

内存：8GB DDR5

存储：60GB SSD

网络：千兆以太网适配器

安装所需空间：50GB

软件环境：

操作系统：CentOS 7

容器平台：Docker 26.1.4

微服务运行时：

JDK：OpenJDK 17

注册中心：Nacos 2.3.2

消息队列：RocketMQ 5.1.4

数据库：MySQL：8.0.31、Redis：7.4

文件存储：FastDFS 6.09

反向代理：FastDFS 内置 Nginx

二、安装与配置 Docker

1. 更新 yum 本地镜像源

```
curl -o /etc/yum.repos.d/CentOS-Base.repo
http://mirrors.aliyun.com/repo/Centos-7.repo
```

2. 安装 yum 工具包

```
yum install -y yum-utils \
    device-mapper-persistent-data \
    lvm2 --skip-broken
```

3. 将软件包信息提前在本地索引缓存，用来提高搜索安装软件的速度。

```
yum makecache fast
```

4. 安装 docker 引擎

```
sudo yum install docker-ce docker-ce-cli containerd.io docker-
buildx-plugin docker-compose-plugin -y
```

5. 启动 docker

systemctl start docker

6. 设置 docker 自启动

systemctl enable docker

7. 测试 docker 是否安装成功

docker version

出现如下内容，即为安装成功

```
[root@localhost conf]# docker version
Client: Docker Engine - Community
 Version:           26.1.4
 API version:       1.45
 Go version:        go1.21.11
 Git commit:        5650f9b
 Built:             Wed Jun  5 11:32:04 2024
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Engine - Community
 Engine:
  Version:          26.1.4
  API version:      1.45 (minimum version 1.24)
  Go version:       go1.21.11
  Git commit:       de5c9cf
  Built:            Wed Jun  5 11:31:02 2024
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          1.6.33
  GitCommit:        d2d58213f83a351ca8f528a95fbd145f5654e957
 runc:
  Version:          1.1.12
  GitCommit:        v1.1.12-0-g51d5e94
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
[root@localhost conf]#
```

图 2-1 docker 版本信息

8. 修改 docker 镜像代理配置文件

vim /etc/docker/daemon.json

加入以下内容：

```
{
    "registry-mirrors": [
        "https://docker.1ms.run",
        "https://docker.xuanyuan.me",
        "https://image.cloudlayer.icu",
        "https://docker-0.unsee.tech",
        "https://docker.tbedu.top",
        "https://hub.rat.dev",
        "https://docker.hlmirror.com",
        "https://docker.imgdb.de",
    "https://docker.m.daocloud.io"
    ]
}
```

9. 修改 docker 配置文件

vim /lib/systemd/system/docker.service

在 ExecStart=后添加配置，远程访问 docker 的端口为 2375

ExecStart=/usr/bin/dockerd -H tcp://0.0.0.0:2375 -H unix://var/run/docker.sock

三、安装与配置 MySQL

1. 拉取 MySQL 镜像容器

   docker pull mysql:8.0.31

2. 启动容器（用户名和密码根据个人需求设置）

   docker run -itd --restart=always --name mysql -p 3306:3306 -e MYSQL_ROOT_PASSWORD=root mysql:8.0.31

3. 导入数据

   使用 navicat 连接 MySQL，分别创建数据库 my_nacos 和 vivaconnect，在对应数据库运行数据库文件，导入数据。数据库文件在白政财毕业设计_源代码文件夹中：my_nacos.sql 和 vivaconnect.sql。

四、安装与配置 Redis

1. 拉取 Redis 镜像容器

   docker pull redis:7.4

2. 启动容器

   docker run -di --restart=always --name=redis -p 6379:6379 redis:7.4

五、安装与配置 Nacos

1. 拉取 Nacos 镜像容器

   docker pull nacos:v2.3.2

2. 创建配置文件夹

   mkdir -p /usr/local/docker/nacos/config

3. 打开配置文件

   vim /usr/local/docker/nacos/config/application.properties

   添加如下内容并修改 MySQL 服务地址：

   # spring

   server.servlet.contextPath=${SERVER_SERVLET_CONTEXTPATH:/nacos}

   server.contextPath=/nacos

   server.port=${NACOS_APPLICATION_PORT:8848}

   server.tomcat.accesslog.max-days=30

   server.tomcat.accesslog.pattern=%h %l %u %t "%r" %s %b %D %{User-Agent}i %{Request-Source}i

```
server.tomcat.accesslog.enabled=${TOMCAT_ACCESSLOG_ENABLED:false}
server.error.include-message=ALWAYS
# default current work dir
server.tomcat.basedir=file:.
#*************** Config Module Related Configurations ***************#
### Deprecated configuration property, it is recommended to use `spring.sql.init.platform` replaced.
spring.datasource.platform=mysql
spring.sql.init.platform=${SPRING_DATASOURCE_PLATFORM:}
nacos.cmdb.dumpTaskInterval=3600
nacos.cmdb.eventTaskInterval=10
nacos.cmdb.labelTaskInterval=300
nacos.cmdb.loadDataAtStart=false
db.num=${MYSQL_DATABASE_NUM:1}
db.url.0=jdbc:mysql://192.168.72.128:3306/my_nacos?characterEncoding=utf8&connectTimeout=1000&socketTimeout=3000&autoReconnect=true&useUnicode=true&useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true
db.user.0=root
db.password.0=root
## DB connection pool settings
db.pool.config.connectionTimeout=${DB_POOL_CONNECTION_TIMEOUT:30000}
db.pool.config.validationTimeout=10000
db.pool.config.maximumPoolSize=20
db.pool.config.minimumIdle=2
### The auth system to use, currently only 'nacos' and 'ldap' is supported:
nacos.core.auth.system.type=${NACOS_AUTH_SYSTEM_TYPE:nacos}
### worked when nacos.core.auth.system.type=nacos
### The token expiration in seconds:
nacos.core.auth.plugin.nacos.token.expire.seconds=${NACOS_AUTH_TOKEN_EXPIRE_SECONDS:18000}
```

```
### The default token:
nacos.core.auth.plugin.nacos.token.secret.key=${NACOS_AUTH_TOKEN:}
### Turn on/off caching of auth information. By turning on this switch, the update of auth information would have a 15 seconds delay.
nacos.core.auth.caching.enabled=${NACOS_AUTH_CACHE_ENABLE:false}
nacos.core.auth.enable.userAgentAuthWhite=${NACOS_AUTH_USER_AGENT_AUTH_WHITE_ENABLE:false}
nacos.core.auth.server.identity.key=${NACOS_AUTH_IDENTITY_KEY:}
nacos.core.auth.server.identity.value=${NACOS_AUTH_IDENTITY_VALUE:}
## spring security config
### turn off security
nacos.security.ignore.urls=${NACOS_SECURITY_IGNORE_URLS:/,/error,/**/*.css,/**/*.js,/**/*.html,/**/*.map,/**/*.svg,/**/*.png,/**/*.ico,/console-fe/public/**,/v1/auth/**,/v1/console/health/**,/actuator/**,/v1/console/server/**}
# metrics for elastic search
management.metrics.export.elastic.enabled=false
management.metrics.export.influx.enabled=false
management.endpoints.web.exposure.include=*
nacos.naming.distro.taskDispatchThreadCount=10
nacos.naming.distro.taskDispatchPeriod=200
nacos.naming.distro.batchSyncKeyCount=1000
nacos.naming.distro.initDataRatio=0.9
nacos.naming.distro.syncRetryDelay=5000
nacos.naming.data.warmup=true
nacos.console.ui.enabled=true
nacos.core.param.check.enabled=true
```

4. 启动容器

```
docker run -d \
  --restart=always \
  --name nacos \
```

```
-e MODE=standalone \
-e JVM_XMS=256m \
-e JVM_XMX=256m \
-e JVM_XMN=256m \
-p 8848:8848 \
-p 9848:9848 \
-v
/usr/local/docker/nacos/config/application.properties:/home/nacos/conf/application.properties \
nacos/nacos-server:v2.3.2
```

5. 验证安装

浏览器输入 192.168.72.128:8848/,出现如下页面即为安装成功

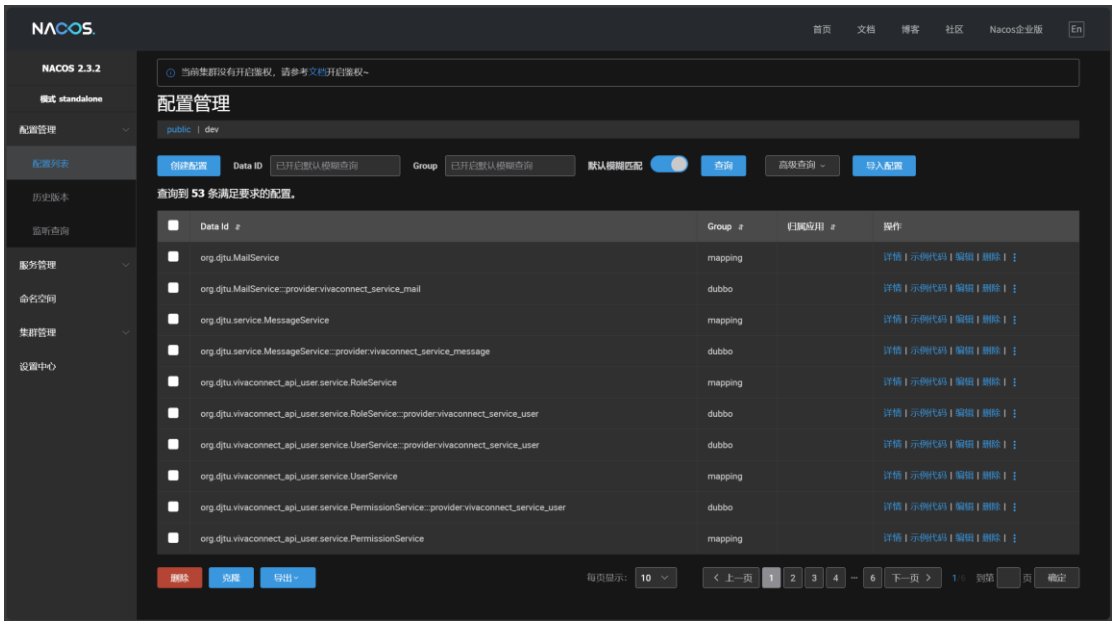

图 5-1 nacos 配置页面

六、 安装与配置 FastDFS

1. 拉取 FastDFS 镜像容器并改名

```
docker pull crpi-qjyjdm4mtqw60x5e.cn-beijing.personal.cr.aliyuncs.com/itbaizhan/fastdfs:1.0
docker pull crpi-qjyjdm4mtqw60x5e.cn-beijing.personal.cr.aliyuncs.com/itbaizhan/fastdfs:1.0
```

2. 启动 tracker 容器

```
docker run -dti  -restart=always --network=host --name tracker --
```

privileged=true -v /var/fdfs/tracker:/var/fdfs -v /etc/localtime:/etc/localtime delron/fastdfs tracker

3. 启动 storage 容器（IP 地址更换为自己的 IP）

   docker run -dti -restart=always --network=host --name storage --privileged=true -e TRACKER_SERVER=192.168.72.128:22122 -v /var/fdfs/storage:/var/fdfs -v /etc/localtime:/etc/localtime delron/fastdfs storage

4. 验证安装

   浏览器访问 192.168.72.128:8888/

   出现如下页面，说明启动成功

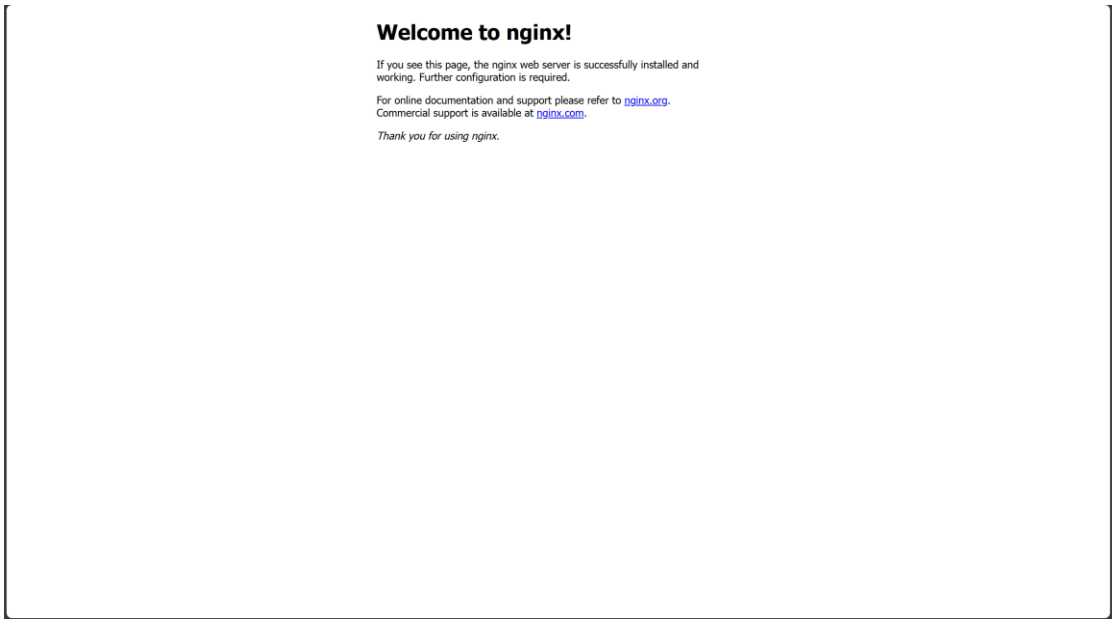

图 6-1 nginx 启动成功页面

七、 安装与配置 RocketMQ

1. 拉取 RocketMQ 镜像容器

   docker pull apache/rocketmq:5.1.4

2. 创建容器共享网络

   docker network create rocketmq

3. 启动 NameServer

   docker run -d --name rmqnamesrv -p 9876:9876 --network rocketmq apache/rocketmq:5.1.4 sh mqnamesrv

   验证启动是否成功

docker logs -f rmqnamesrv

有如下信息，即代表启动成功

'The Name Server boot success..'

4. 创建配置文件夹并赋予读写权限

mkdir -p /usr/local/docker/rocketMQ/

mkdir -p /usr/local/docker/rocketMQ/timer_wheel_store

chmod -R 777 /usr/local/docker/rocketMQ/timer_wheel_store

mkdir -p /usr/local/docker/rocketMQ/timer_checkpoint

chmod -R 777 /usr/local/docker/rocketMQ/timer_checkpoint

5. 创建配置文件夹

在/usr/local/docker/rocketMQ/目录下创建并修改文件 broker.conf

vim ./broker.conf

写入如下内容（IP 地址改为自己的 IP）：

brokerIP1=192.168.72.128

traceTopicEnable=true

enableTimerMessage=true

timerWheelStorePath=/usr/local/docker/rocketMQ/timer_wheel_store

timerCheckpointPath=/usr/local/docker/rocketMQ/timer_checkpoint

4. 启动 Broker+Proxy

```
docker run -d \
--restart=always \
--name rmqbroker \
--network rocketmq \
-p 10912:10912 -p 10911:10911 -p 10909:10909 \
-p 8080:8080 -p 8081:8081 \
-e "NAMESRV_ADDR=rmqnamesrv:9876" \
-v ./broker.conf:/home/rocketmq/rocketmq-5.1.4/conf/broker.conf \
apache/rocketmq:5.1.4 sh mqbroker --enable-proxy \
-c /home/rocketmq/rocketmq-5.1.4/conf/broker.confe
```

5. 安装 RocketMQ 可视化面板

5.1 拉取镜像并改名

docker pull crpi-qjyjdm4mtqw60x5e.cn-beijing.personal.cr.aliyuncs.com/itbaizhan/rocketmq-dashboard

docker tag crpi-qjyjdm4mtqw60x5e.cn-

beijing.personal.cr.aliyuncs.com/itbaizhan/rocketmq-dashboard

apacherocketmq/rocketmq-dashboard

5.2 启动容器（IP 地址改为自己的 IP）

docker run -d --name rocketmq-dashboard --net rocketmq -e "JAVA_OPTS=-Drocketmq.namesrv.addr=192.168.72.128:9876" -p 8899:8080 -t apacherocketmq/rocketmq-dashboard:latest

6. 验证安装

浏览器输入 192.168.72.128:8899

出现如下页面，且无报错，即为安装成功

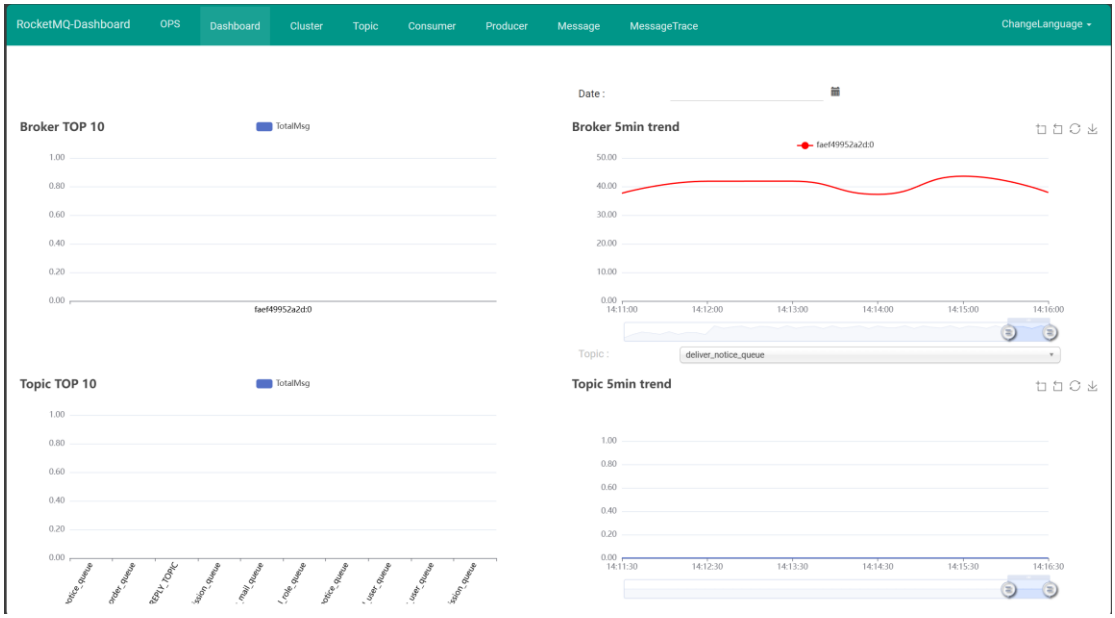

图 7-1 rocketmq 可视化面板

7. 添加主题

在可视化面板页面，点击 Topic，点击新增，新增主题：sync_user_queue、del_user_queue、sync_role_queue 、 del_role_queue 、 del_permission_queue 、 deliver_notice_queue 、delay_notice_queue、check_order_queue、send_alarm_mail_queue 和 send_status_mail_queue。

八、修改后端项目配置

将 vivaconnect 项目文件中 web 和 service 模块下所有 bootstrap.yaml 文件中 server-addr 和 namespace 的值改为注册中心的值。并删除如下内容：

config:

import: file:E:\Project\vivaconnect\vivaconnect_commons\common-config.yaml

同时删除 vivaconnect_commons/common-config.yaml 文件。